

Programmation dynamique

I. Introduction : Le problème du rendu de monnaie

1. Retour sur l'algorithme glouton

On utilise l'algorithme glouton pour trouver une réponse rapide (pas forcément exacte) à un problème np complet (ne pouvant pas être résolu avec un algorithme de complexité polynomiale, mais dont une solution donnée est facilement vérifiable). L'algorithme glouton peut échouer à trouver une réponse.

Par exemple : rendu de monnaie glouton (rendre la monnaie avec un nombre minimal de pièces)

Système euros : euros = [50, 20, 10, 5, 2, 1] Le système est canonique et le rendu de monnaie est optimal rendu(euros,48) renvoie 20,20,5,2,1	ancien système impérial britannique imperial = [30, 24, 12, 6, 3, 1] rendu(imperial,48) renvoie 30,12,6 Ce rendu n'est pas optimal (on aurait pu le faire en deux pièces de 24,24)
--	--

2. Recherche de la réponse optimale

L'algorithme glouton peut échouer à trouver une réponse, et s'il en trouve une elle n'est pas forcément optimale.

Une idée est d'utiliser la récursivité :

soit x une valeur de l'une des pièces du système monétaire.

Pour rendre une somme s de façon optimale, si l'on veut utiliser au moins une fois la pièce x , il suffit de prendre la pièce x et la somme s-x de façon optimale.

Il suffit de prendre le minimum de pièce utilisées pour toutes les valeurs des pièces.

Relation récursive : (j'appelle f(s) le nombre minimal de pièces qu'il faut utiliser pour payer la somme s)

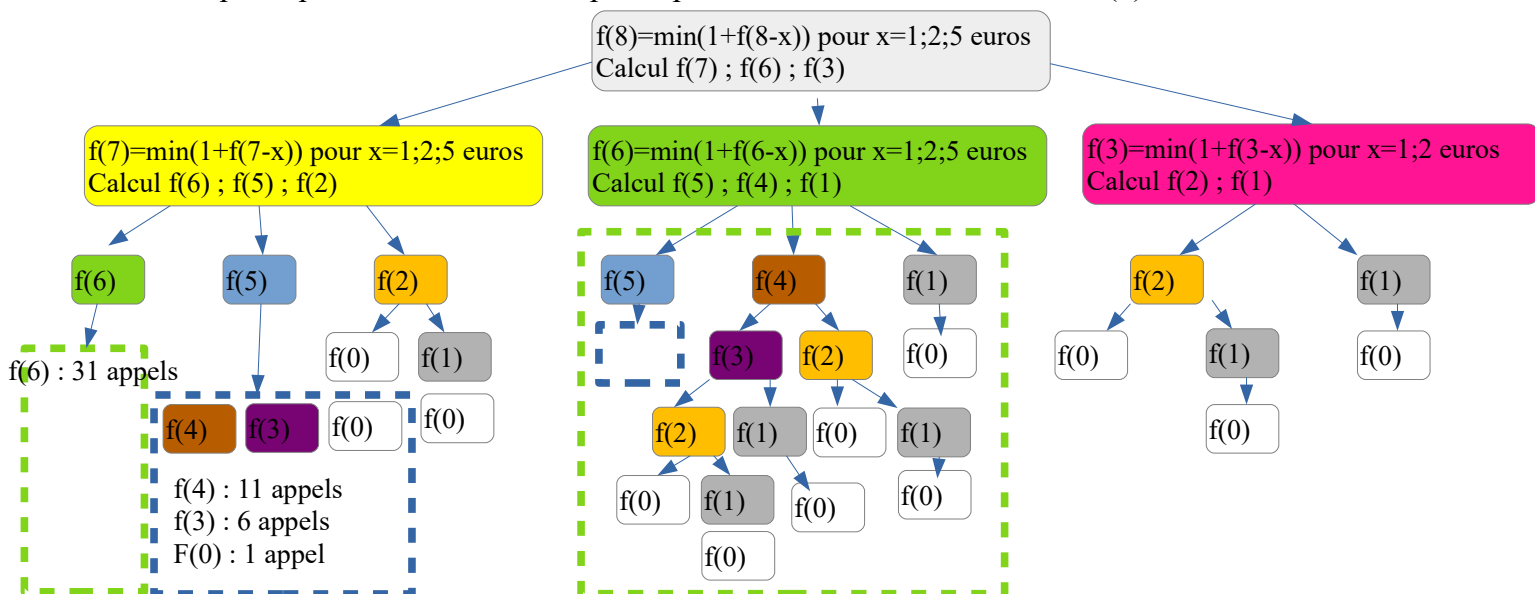
$f(0)=0$ (cas limite)

et $f(s) = 1 + \min_{x \leq s} (f(s-x))$,

```
def recur_monnaie(systeme,somme) :
    si somme = 0 :
        renvoyer 0
    sinon:
        renvoyer
        1+ mini(recur_monnaie(systeme, somme - x))
            pour x <=somme
```

Remarques :

- cette version ne donne pas la liste des pièces mais simplement le nombre de pièces.
- fonctionne bien pour les petites sommes
- n'est pas capable de donner une réponse pour 48 raison : voir le calcul de f(8) ci-dessous.



On remarque qu'on recalcule de nombreuses fois le même terme .

En tout pour $f(8)$: $31+18+4 + 31 + 6 = 90$ appels

La complexité est exponentielle ; les calculs ne sont pas indépendants,

3. Réponse optimale plus rapide :

Une idée classique consiste à **mémoriser les résultats** des appels pour être sûr qu'on n'aura pas besoin de les calculer plusieurs fois.

On va donc créer un tableau conservant ces données, et calculer de façon systématique pour des valeurs croissantes de l'index. (plus de récursivité).

```
def dynam_monnaie(systeme, somme):
    nb = [0] * (somme + 1) # init tableau
    Pour s allant de 1 à somme:
        nb[s] = s # au pire on rend s pièces de 1
        pour valeur dans systeme :
            si valeur <= s :
                nb[s] = min(nb[s], 1 + nb[s - valeur])
```

Remarque :

- Coût en mémoire : $somme+1$ emplacements pour un entier (64 bits par défaut),
- Complexité : deux boucles imbriquées : $somme \times valeur$ (donc plus du tout exponentiel),
- l'algorithme proposé ne donne pas la liste des pièces, simplement leur nombre. Pour répondre à cet inconvénient, on rajoute un deuxième tableau qui, pour chaque somme s , donne la liste des pièces,

A faire :

Tester cet algorithme « à la main » sur l'appel `dynam_monnaie([10, 5, 2, 1], 8)`, puis vérifier en saisissant le script dans [pythontutor](https://pythontutor.com).

4. Conclusion

On peut dire que la résolution algorithmique d'un problème relève de la programmation dynamique si

- ▷ le problème peut être résolu à partir de sous-problèmes similaires mais plus petits ;
- ▷ l'ensemble de ces sous-problèmes est discret, c'est-à-dire qu'on peut les indexer et ranger les résultats dans un tableau ;
- ▷ la solution optimale au problème posé s'obtient à partir des solutions optimales des sous-problèmes ;
- ▷ les sous-problèmes ne sont pas indépendants et un traitement récursif fait apparaître les mêmes sous-problèmes un grand nombre de fois.

5. Comparaison

Glouton s'utilise pour chercher une solution rapide (pas forcément optimisé) à un problème np complet, selon un critère de choix.

La récursivité (diviser pour régner) s'utilise quand un problème peut se couper en deux (ou plus) sous problèmes indépendants. La solution se retrouve en recombinaison des solutions de chaque sous problème.

La programmation dynamique s'applique quand les sous-problèmes se recoupent, c'est-à-dire lorsque les sous-problèmes ont des problèmes communs. Un algorithme de programmation dynamique résout chaque sous-problème une seule fois et mémorise sa réponse dans un tableau, évitant ainsi le recalcul de la solution.

6. Reconstitution de la solution

On utilise la **mémoïsation** : il s'agit d'une solution récursive optimisée à un problème en mémorisant les solutions dans un tableau. C'est une sorte de mix entre le diviser pour régner et la programmation dynamique.

Si on remplace la dernière ligne de la fonction `dynam_monnaie` par `return nb`, l'appel `dynam_monnaie(imperial, 17)` renvoie le tableau
`[0, 1, 2, 1, 2, 3, 1, 2, 3, 2, 3, 4, 1, 2, 3, 2, 3, 4]`.

Il y a donc une réponse optimale avec 4 pièces, comment la reconstituer ?

On a `nb[17] = 4`, on cherche une pièce `x` telle que `nb[17 - x] = 3`.

On a le choix entre `x = 1`, `x = 3` et `x = 12`. Choisissons `x = 12`.

Donc, `nb[17] = 1 + nb[17 - 12] = 1 + nb[5]` : pour rendre une somme égale à 17, on rend **une pièce de 12** et il reste à rendre une somme de 5.

On cherche maintenant une pièce `x'` telle que `nb[5 - x'] = 2`, on peut choisir `x' = 3` (`x' = 1` aurait également convenu). Ainsi :

- `nb[5] = 1 + nb[2]` : pour rendre 5, on rend **1 pièce de 3** et il reste à rendre une somme de 2,
- `nb[17] = 1 + nb[5] = 1 + 1 + nb[2] = 2 + nb[2]`

Puis

- `nb[2] = 1 + nb[1]` : pour rendre 2, on rend **1 pièce de 1** et il reste à rendre une somme de 1.
- `nb[17] = 2 + nb[2] = 2 + 1 + nb[1] = 3 + nb[1]`
- `nb[1] = 1` : pour rendre 1, on rend **1 pièce de 1**.

Une solution optimale est donc de payer $17 = 1 + 1 + 3 + 12$, avec 4 pièces.

On peut modifier la fonction précédente pour reconstituer une décomposition optimale : il suffit de détailler le calcul du minimum et en profiter pour mémoriser les valeurs notées `x`, `x'` etc. ci-dessus.

```
1. from math import inf
2. def dynMemoiseReconstitue(valeursFaciales, somme):
3.     f = [0] * (somme + 1)
4.     g = [0] * (somme + 1)
5.     for s in range(1, somme + 1):
6.         f[s] = inf
7.         for x in valeursFaciales:
8.             if s >= x:
9.                 if 1 + f[s - x] < f[s]:
10.                    f[s] = 1 + f[s - x] # mise à jour du minimum
11.                    g[s] = x # pièce à rendre pour une somme s
12.
13.     monnaie = []
14.     s = somme
15.     while s > 0 :
16.         monnaie.append(g[s])
17.         s = s - g[s]
18.     return monnaie
```

Cliquer [ici](#) pour tester la solution sous pythontutor.

https://pythontutor.com/render.html#code=from%20math%20import%20inf%0A%0Adef%20dynMemoiseReconstitue%28valeursFaciales,%20somme%29%3A%0A%20%20%20%20f%20%3D%20%5B0%5D%20*%20%28somme%20%2B%201%29%0A%20%20%20%20g%20%3D%20%5B0%5D%20*%20%28somme%20%2B%201%29%0A%20%20%20%20for%20s%20in%20range%281,%20somme%20%2B%201%29%3A%0A%20%20%20%20%20%20%20%20%20f%5Bs%5D%20%3D%20inf%0A%20%20%20%20%20%20%20%20%20for%20x%20in%20valeursFaciales%3A%0A%20%20%20%20%20%20%20%20%20%20if%20s%20%3E%3D%20x%3A%0A%20%20%20%20%20%20%20%20%20%20if%201%20%2B%20f%5Bs%20-%20x%5D%20%3C%20f%5Bs%5D%3A%0A%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20f%5Bs%5D%20%3D%201%20%2B%20f%5Bs%20-%20x%5D%20%23%20mise%20%C3%A0%20jour%20du%20minimum%0A%20g%5Bs%5D%20%3D%20x%20%23%20pi%C3%A8ce%20%C3%A0%20rendre%20pour%20une%20somme%20s%0A%20%0A%20%20%20%20monnaie%20%3D%20%5B%5D%0A%20%20%20%20s%20%3D%20somme%0A%20%20%20%20while%20s%3E0%20%3A%0A%20%20%20%20%20%20%20%20%20monnaie.append%28g%5Bs%5D%29%0A%20%20%20%20%20%20%20%20%20s%20%3D%20s%20-%20g%5Bs%5D%0A%20%20%20%20return%20monnaie%0A%20%20%20%20%0Aprint%28dynMemoiseReconstitue%28%5B12,%206,%203,%201%5D,%2017%29%29&cumulative=false&curInstr=0&heapPrimitives=nevernest&mode=display&origin=opt-frontend.js&py=3&rawInputLstJSON=%5B%5D&textReferences=false